Vision Robotique - TP1 - CImg

L'objectif de ce TP est la mise en oeuvre de la bibliothèque CImg pour le traitement des images. Les documents de référence pour cette bibliothèque se trouveront dans l'espace partagé dédié aux TP du cours de Vision sur la plateforme pédagogique, ainsi que sur le net : http://cimg.eu/

CMake & Make

Pour construire l'exécutable de notre programme et des suivants, nous allons utiliser un système de construction logiciel : **CMake**.

En premier lieu, nous créons notre répertoire projet

> mkdir tp1-vision

et un sous-répertoire où on rangera nos codes sources :

> cd tp1-vision
> tp1-vision$mkdir src

Hello world … Lena

On commence avec le plus simple des codes source, le fameux "hello world", à entrer dans un fichier *hello.cpp* placé dans le dossier *src* en utilisant l'éditeur texte de votre choix (nano, vi, emacs, gedit …). Une fois que notre espace de travail sera configuré à l'étape suivante, on pourra bien sûr utiliser notre EDI préféré.

```
#include <iostream>
#include "libcimg/CImg.h"
using namespace cimg_library;

int main(int argc,char **argv) {
  // Read image filename from the command line (or set it to "lena.jpg" if option '-i' is not pro
vided)
  const char* file_i = cimg_option("-i","lena.jpg","Input image");

  // Load an image
  CImg<unsigned char> image = CImg<>(file_i);

  unsigned char purple[] = {255,0,255};// Define a purple color
  image.draw_text(10,10,"Hello World",purple);//Write text on image
  image.display("lena");//Display image
  return 0;
}
```

Dans le répertoire projet, on peut alors créer notre fichier CMakeLists.txt :

```
#Déclaration du projet
project(tp1-vision)
set(EXECUTABLE_OUTPUT_PATH bin/${CMAKE_BUILD_TYPE})
include_directories(include)
#Déclaration de l'exécutable
add_executable(hello src/cimg_hello.cpp)
```

```
#Configuration de l'édition de liens
target_link_libraries(hello pthread X11)
```

À partir du fichier descriptif du projet (CMakeLists.txt) et des informations sur la configuration logicielle de la machine effectuant la compilation (listées dans le fichier CMakeCache.txt, lequel sera prérempli lors de la phase d'analyse de la configuration), CMake peut générer les fichiers suivants :

- un fichier Makefile
- un fichier de projet d'EDI
- ou une combinaison des deux pour les EDI supportant les fichiers Makefile; c'est la cas de l'EDI CodeBlocks que nous allons utiliser ici.

Pour obtenir la liste des générateurs de projet d'EDI :

> cmake –help

Dans notre situation :

> cmake . -G'CodeBlocks - Unix Makefiles'

Il est alors possible de lancer un build de notre code à l'aide de la commande

> make

dans le répertoire du projet, ou en lancant le build de notre projet à partir de notre EDI CodeBlocks. **Avant cela**, nous allons indiquer que nous souhaitons que la construction se fasse en mode *Debug* et positionnons la variable *CMAKE_BUILD_TYPE* dans le fichier CMakeCache.txt :

```
CMAKE_BUILD_TYPE:STRING=Debug
```

Nous pouvons maintenant exécuter le programme.

> tp1-vision/bin/Debug$./hello lena.jpg

A noter que notre code prend un fichier image en argument, ici *lena.jpg*, que l'on trouvera dans notre espage partagé.

Optionnel : Pour voir apparaître CMakeCache.txt dans le répertoire virtuel CMake Files de CodeBlocks, rajouter avant la fin du fichier nom_de_projet.cbp :

```
<Unit filename="/home/senn/work_C/td1-mod/CMakeCache.txt">
            <Option virtualFolder="CMake Files\"/>
        </Unit>
```

LIVRABLE 1
Programmer *Hello World Lena* tel que décrit ci-dessus.

Opérations élémentaires

Niveau 1

On se propose de prendre en main la bibliothèque **CImg**, en commençant par quelques

opérations élémentaires.

LIVRABLE 2
A partir du programme *hello world* précédent.
Réaliser un fort bruitage de l'image à l'aide de la fonction *noise*.
Réaliser un débruitage de l'image à l'aide d'un filtre median (fonction *blur_median*).
Afficher l'image.

On donne ci-dessous deux versions du code pour réaliser ces premières opérations. La version *pipelinée* permet une écriture plus dense du code mais le rend moins lisible, et difficilement commentable. Si les performances du code ne sont pas meilleures, il n'y a donc pas lieu de l'écrire de la sorte.

```
//NOT PIPELINED
image.draw_text(10,10,"Hello World",purple);
image.noise(10,2);
image.blur_median(5,0);
image.display("lena");

//PIPELINED
image.draw_text(10,10,"Hello World",purple).noise(10,2).blur_median(5,0).display("lena");
```

Remarque : Note that most of the CImg functions have two versions : one that acts in-place (which is the case of *noise* or *blur_median* for instance), and one that returns the result as a new image (the name of the function begins then with **get_**). In this case, one could have also written `image = image.get_noise(10,2);` (more expensive, since it needs an additional copy operation).

Niveau 2

On se propose maintenant d'essayer d'autres filtres pour débruiter l'image et à chaque fois d'estimer la qualité du traitement, de mesurer la performance (fonction *variance*, *variance_noise*), et de comparer au final les résultats.

LIVRABLE 3
Mettre en oeuvre les filtres *blur_median*, *blur_box*, *blur_anisotropic*.
Afficher les images résultats de chacun d'eux.
Mesurer et afficher leurs performances.

Si l'on souhaite afficher plusieurs images à l'écran, il est nécessaire pour chacune d'elles de créer un *display* distinct en utilisant la classe *CImgDisplay* comme indiqué dans l'exemple de code ci-dessous.
On note l'utilisation des fonctions *variance...* :

Précisez ce que vous mesurer lorsque vous utilisez ces deux fonctions.

```
cout << "bruitage" << endl;
CImg<unsigned char> image_noised = image.noise(10,2);
cout << "variance =" << image_noised.variance(0) << endl;
cout << "variance noise =" << image_noised.variance_noise(0) << endl;
CImgDisplay disp_noise(image,"noise");
image.display(disp_noise);
```

En effet, appeler la fonction *display* à partir de la classe *CImg* ouvre une fenêtre de visualisation

interactive :

- dont la taille est plus grande que celle de l'image,
- qui reste à l'écran et place le programme dans l'attente de l'appuie sur la touche "ESC",
- qui disparaît une fois cette touche appuyée.

Afficher une image dans un display permet entre autres :

- de visualiser l'image avec ses dimensions originales,
- d'ouvrir plusieurs displays simultanément,
- de fermer ceux que l'on souhaite,
- de gérer comme on l'entend les interactions avec les évènements des périphériques d'entrée (clavier, souris ...).
  Tous les displays seront fermés à la terminaison du programme. Il sera donc nécessaire d'attendre une commande de l'utilisateur pour quitter l'exécution. La bibliothèque **CImg** contient toutes les fonctions nécessaires à la prise en compte des évènements systèmes et notamment provenant des périphériques d'entrée (*CImgDisplay.is_key()*; *CImgDisplay.button()*; *CimgDisplay.mouse_x()*; *CImgDisplay.wait()*; ...) .

Dans le cas présent, l'utilisation de **cin.ignore();** ou **cin.get();** à la fin du programme peut suffire.

Exemple tiré du manuel de référence :

```
CImgDisplay disp(400,400);
const char *const keycode = "TAB";
while (!disp.is_closed()) {
  if (disp.is_key(keycode)) { ... }  // Equivalent to 'if (disp.is_keyTAB())'
  disp.wait();
}
```

Segmentation par histogramme

Histogramme

Un grand nombre d'algorithmes en traitement d'images sont basés sur la séparation des pixels d'une image en classes selon leur niveau. La représentation du compte du nombre de pixels à chaque niveau est l'histogramme de l'image. L'exemple de code ci-dessous illustre le calcul et l'affichage de l'histogramme d'une image de différentes façons :

```
CImgDisplay disp(grayimage,"display");
grayimg.display(disp);//affichage de l'image en niveaux de grix


//méthode 1, à partir de la classe CImg
CImg<float> imghisto = grayimg.histogram(256);//calcul de l'histogramme
imghisto.display_graph(0,3);//affichage de l'histogramme, dans une fenêtre interactive, donc bloq
uante

//méthode 2, à partir de la classe CImgDisplay
CImgDisplay hist;
img.display_graph(hist,3,2,"gray levels",0,0,"count",0,0,false);//affichage de l'histogramme, enc
ore dans une fenêtre interactive, bloquante, mais persistente

//méthode 3, avec draw_graph à partir de la classe CImg
```

```
CImg<unsigned char> visu(500,400,1,3,0);//on construit l'image
CImgDisplay draw_disp(visu,"histogramme");//on construit son display et on l'affiche
const unsigned char red[] = { 255,0,0 };//on définit une couleur
visu.fill(0).draw_graph(grayimg.histogram(256),red,1,3,0,255,0).display(draw_disp);//ATTENTION ic
i on écrase l'image "grayimg" avec l'image de l'histogramme
//OU
visu.fill(0).draw_graph(grayimg.get_histogram(256).normalize(0,255),red,1,3,0,255,0).display(draw
_disp);//ICI on n'écrase pas "grayimg", il faut cependant normaliser les valeurs retournées par "
get_histogram"
```

Les fonctions données ci-dessous permettent la transformation d'une image couleur RGB en une image en niveaux de gris.

```
//================================================
//functions prototypes

//rgb2gray : the input is a RGB image, the ouput is a graylevel image
//a pixel gray level is the mean of its R,G and B values
CImg<unsigned char> rgb2gray(CImg<unsigned char>);

//rgb2grayw : the input is a RGB image, the ouput is a graylevel image
//a pixel gray level is a combination ot its R,G, and B values such as
//graylevel=(0.299*R + 0.587*G + 0.114*B);
CImg<unsigned char> rgb2grayw(CImg<unsigned char>);

//================================================
//functions

//*****************************************************************
//rgb2gray : the input is a RGB image, the ouput is a graylevel image
CImg<unsigned char> rgb2gray(CImg<unsigned char> prem)
{
CImg<unsigned char> gray(prem.width(), prem.height(), 1, 1, 0);
// for all pixels x,y in image
  cimg_forXY(prem,x,y) {
    // Separation of channels
    int R = (int)prem(x,y,0,0);
    int G = (int)prem(x,y,0,1);
    int B = (int)prem(x,y,0,2);
    // Arithmetic addition of channels for gray
    int grayValue = (int)(0.33*R + 0.33*G + 0.33*B);
    // Real weighted addition of channels for gray
    //int grayValueWeight = (int)(0.299*R + 0.587*G + 0.114*B);
    // saving pixel values into image information
    gray(x,y,0,0) = grayValue;
    //grayWeight(x,y,0,0) = grayValueWeight;
}
return gray;
}
//*****************************************************************


//*****************************************************************
//rgb2grayw : the input is a RGB image, the ouput is a graylevel image
CImg<unsigned char> rgb2grayw(CImg<unsigned char> prem)
{
CImg<unsigned char> gray(prem.width(), prem.height(), 1, 1, 0);
// for all pixels x,y in image
```

```
  cimg_forXY(prem,x,y) {
    // Separation of channels
    int R = (int)prem(x,y,0,0);
    int G = (int)prem(x,y,0,1);
    int B = (int)prem(x,y,0,2);
    // Arithmetic addition of channels for gray
    //int grayValue = (int)(0.33*R + 0.33*G + 0.33*B);
    // Real weighted addition of channels for gray
    int grayValue = (int)(0.299*R + 0.587*G + 0.114*B);
    // saving píxel values into image information
    gray(x,y,0,0) = grayValue;
    //grayWeight(x,y,0,0) = grayValueWeight;
}
return gray;
}
//****************************************************************
```

LIVRABLE 4
Transformer l'image *Lena.jpg* en niveaux de gris en utilisant les deux variantes.
Afficher ces deux images.
Afficher leur histogramme.

Comparer les deux images en niveaux de gris obtenues. Laquelle préférez-vous ?

Seuillage

On sépare en deux classes les pixels de l'image en niveaux de gris à partir d'un seuil : on obtient une image binaire. Pour cela on utilise la fonction **.get_threshold()** à partir de la classe *CImg*.

```
CImg<unsigned char> res;//on déclare une image
CImgDisplay dispr;//on déclare un display
int seuil = 128;//on déclare un seuil et on lui donne une valeur
res = grayimg.get_threshold(seuil).normalize(0,255);//on binarise l'image grayimg
dispr.display(res).set_title("binarized");//on affiche l'image binarisée dans le display auquel o
n donne un titre
```

On souhaite rendre interactif notre programme en sélectionnant la valeur du seuil à partir de la position de la souris dans la fenêtre qui montre l'histogramme. Pour cela, on écrit la boucle suivante :

```
//calcul histogramme et affichage dans la fenetre de visualisation
visu.fill(0).draw_graph(grayimg.get_histogram(256).normalize(0,255),red,1,3,0,255,0).display(draw
_disp);

while (!disp.is_closed() && !draw_disp.is_keyQ() && !draw_disp.is_keyESC())
{
    //affichage de la valeur du seuil en jaune dans la deuxième fenetre de visualisation
    visu2.fill(0).draw_text(50,5,"seuil = %i",yellow,0,1,24,seuil).display(draw_disp2);
    res = grayimg.get_threshold(seuil).normalize(0,255).draw_text(50,5,"seuil = %i",blue,0,1,24,s
euil);//on binarise l'image et on écrit dedans la valeur du seuil
    dispr.display(res).wait(20).set_title("binarized");//on affiche l'image binarisée et son titr
e
    if (draw_disp.button() && draw_disp.mouse_x()>=0)//si un boutton souris pressé et la souris e
st dans la fenetre de l'histo
     seuil= ((draw_disp.mouse_x() * 255) / visu.width());//on calcule une nouvelle valeur pour le
```

```
  seuil depuis la position
      //de la souris
 }//fin boucle while
```

LIVRABLE 5
A partir de la boucle suivante, écrire le reste du programme qui
Affiche l'image "lena.jpg" convertie en niveaux de gris
Affiche son histogramme
Affiche l'image binarisée pour la valeur du seuil fixée par l'utilisateur à l'aide de la position de la souris

Pourquoi a-t-on besoin d'une seconde fenêtre de visualisation pour afficher la valeur du seuil ?
Quelles sont les valeurs du seuil qui vous permettent de reconnaître lena dans l'image binarisée ?